VOICE-ACTUATED TRACKING SYSTEM FOR THE TWIN ROTOR PLATFORM

By

E/21/338 RISWAN A.L.M.
E/21/205 SHATHURSHIGA J.
E/21/100 DILSHAN P.D.

ME 325 – MECHANICAL ENGINEERING GROUP PROJECT

presented in partial fulfillment of the requirements
for the degree of B.Sc. Engineering

University of Peradeniya
Date of submission: [14th August, 2026]

Supervised by:
Dr. D. H. S. Maithripala, Department of Mechanical Engineering
Faculty of Engineering

DECLARATION

We hereby declare that the work presented in this report, titled "Voice-Actuated Tracking System for the Twin Rotor Platform," is our own original work carried out under the supervision of Dr. D.H.S. Maithripala, Department of Mechanical Engineering, Faculty of Engineering, University of Peradeniya, in partial fulfillment of the requirements of the ME 325 Mechanical Engineering Design Project. Any material drawn from external sources has been duly acknowledged and referenced. This report has not been submitted, in whole or in part, for any other academic qualification at this or any other institution.

Signed:

E/21/338 Riswan A.L.M. _____ Date: _____

E/21/205 Shathurshiga J. _____ Date: _____

E/21/100 Dilshan P.D. _____ Date: _____

APPROVAL / CERTIFICATION

This is to certify that the project report titled "Voice-Actuated Tracking System for the Twin Rotor Platform," submitted by E/21/338 Riswan, E/21/205 Shathurshiga and E/21/100 Dilshan has been carried out under my supervision and is approved for submission in partial fulfillment of the requirements for the degree of B.Sc. Engineering, Department of Mechanical Engineering, Faculty of Engineering, University of Peradeniya.

Supervisor: Dr. D.H.S. Maithripala

Signature: _____ Date: _____

Head of the Department: Dr. W.P.D. Fernando

Signature: _____ Date: _____

External Examiner: _____

Signature: _____ Date: _____

ACKNOWLEDGMENTS

We would like to express our sincere gratitude to our supervisor, Dr. D. H. S. Maithripala, Department of Mechanical Engineering, Faculty of Engineering, University of Peradeniya, for his invaluable guidance, patience and encouragement throughout the course of this project. His insightful feedback and willingness to share his deep knowledge of control systems and rotor dynamics helped us navigate the many technical challenges we encountered, from the initial system design to the final hardware bring-up, and pushed us to think more critically and rigorously at every stage. We are especially thankful for the freedom he gave us to explore our own ideas while ensuring we stayed grounded in sound engineering practice, and for the countless lessons, both technical and otherwise, that we will carry forward well beyond this project.

We would also like to thank the Department of Mechanical Engineering, University of Peradeniya, for providing the laboratory facilities and the Twin Rotor MIMO System platform that made this project possible, as well as our families and friends for their continued support and encouragement throughout this endeavour.

ABSTRACT

This report presents the design, development and completion of TwinTalk, a voice-actuated tracking and control system for a laboratory Twin Rotor MIMO System (TRMS) platform. The project addresses the practical difficulty operators face when manually issuing setpoint commands to a two-degree-of-freedom rotor testbed, by replacing manual entry with natural-language voice commands that are interpreted, validated and executed through a human-supervised pipeline. The system architecture links a mobile application, a speech-to-intent module, a multi-agent AI orchestrator built with LangGraph, a MeshCat-based three-dimensional kinematic simulation, and a Raspberry Pi hardware interface that drives the pitch and yaw rotors through pulse-width-modulated signals. Central to the design is a Human-in-the-Loop (HiL) safety gate: no command is written to the physical hardware until the operator has reviewed a simulated preview of the resulting motion and has explicitly authorized it by voice.

The complete software pipeline was verified end-to-end during the mid-term phase, and the remaining hardware phase of the project has since been completed. The Raspberry Pi was connected to the physical twin-rotor actuators and [describe handshake/PWM smoke-test outcome]. A linearized PID controller was tuned and brought up as the closed-loop baseline, achieving [settling time / overshoot / steady-state error figures]. This baseline was subsequently replaced with a nonlinear SE(3) geometric controller, which [describe observed improvement, e.g. for large-angle manoeuvres]. Gemini and Whisper were benchmarked for speech recognition under realistic rotor noise, with [chosen provider] selected as the final ASR back-end based on [accuracy/latency rationale].

An end-to-end voice-to-rotor demonstration was recorded, in which [brief description of what the demo showed, e.g. success rate or example commands executed]. The completed project demonstrates that natural voice interaction, combined with a simulation-backed safety gate and nonlinear geometric control, can meaningfully lower the barrier to operating aerospace-representative hardware while preserving full operator authority over every physical action.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF SYMBOLS AND ABBREVIATIONS

Symbol / Abbreviation	Description
AI	Artificial Intelligence
ASR	Automatic Speech Recognition
HiL	Human-in-the-Loop
LLM	Large Language Model
PID	Proportional–Integral–Derivative (controller)
PWM	Pulse-Width Modulation
SE(3)	Special Euclidean group of rigid-body motions in three dimensions
TRMS	Twin Rotor MIMO System
θ	Pitch angle of the twin-rotor body
ψ	Yaw angle of the twin-rotor body

CHAPTER 1: INTRODUCTION

1.1 Background

The Twin Rotor MIMO System (TRMS) is a widely used laboratory testbed for the study of nonlinear, coupled, multi-input multi-output flight-representative dynamics. It consists of a pivoted beam carrying a main rotor and a tail rotor at either end, together approximating the pitch and yaw behaviour of a helicopter. Because the two rotors are aerodynamically and mechanically coupled, the TRMS is routinely used to develop and benchmark control strategies ranging from classical PID control to advanced nonlinear and geometric methods before they are applied to full-scale rotorcraft or unmanned aerial vehicles (UAVs).

In most laboratory settings, operators interact with the TRMS through manual setpoint entry — typing pitch and yaw targets into a control interface or MATLAB/Simulink model. This is workable for controlled experiments but becomes a bottleneck when rapid, hands-free interaction is desirable, for example during live demonstrations, multi-tasking operation, or when the operator's hands are occupied with other equipment. Recent advances in speech recognition and large language model (LLM) based agents make it possible to replace manual entry with natural spoken commands, provided that appropriate safeguards are put in place before any command reaches physical hardware.

1.2 Problem Statement

There is currently no integrated, safety-conscious pipeline that allows an operator to control a twin-rotor platform through natural voice commands while retaining full authority over what the hardware ultimately does. A naïve voice-to-actuator pipeline risks executing misinterpreted commands directly on hardware, which is unacceptable for a coupled, nonlinear aerospace-representative system where large-angle manoeuvres can be physically damaging if incorrectly commanded.

1.3 Objectives

The primary objective of this project is to develop a modular, voice-actuated control system for the Twin Rotor V1.0 platform that enables hands-free trajectory tracking. This is achieved by integrating speech recognition with both linear (PID) and nonlinear (geometric, SE(3)) control, and by creating a professional bridge between natural human interaction and complex aerospace hardware. The specific objectives are:

- Replace manual setpoint entry with natural speech commands that are parsed and validated before execution.
- Develop a multi-agent AI orchestrator that interprets operator intent and produces precise pitch/yaw trajectory commands.
- Enforce Human-in-the-Loop (HiL) safety, such that no command reaches the rotors without an explicit, simulation-backed operator approval.
- Move beyond linear PID control to nonlinear SE(3) geometric control for robust large-angle manoeuvres.

1.4 Scope

The scope of the project covers the mobile application, the speech-to-intent and AI orchestration layers, a three-dimensional kinematic simulation used as a safety preview, and the hardware control layer that ultimately drives the physical TRMS via a Raspberry Pi. The project targets the existing laboratory Twin Rotor V1.0 unit and does not extend to airframe redesign or to rotor/actuator replacement; the physical hardware is treated as fixed and the contribution of the project lies in the software and control layers built around it.

1.5 Report Structure

Chapter 2 reviews relevant literature on TRMS control, geometric control and voice/LLM-based orchestration. Chapter 3 describes the overall system design and methodology, including the software architecture and the Human-in-the-Loop safety mechanism. Chapter 4 details the implementation carried out to date. Chapter 5 presents the experiments, results and mid-term progress achieved. Chapter 6 concludes the report and gives recommendations, and Chapter 7 outlines the remaining scope of work for the second half of the project.

CHAPTER 2: LITERATURE REVIEW

2.1 Twin Rotor MIMO System Control

The TRMS is a standard benchmark plant in control engineering education and research because its two rotor axes are cross-coupled through aerodynamic reaction torques, making single-axis, decoupled control strategies inadequate for large or fast manoeuvres. Classical approaches linearize the system about an operating point and apply PID or state-feedback control, which perform adequately for small deviations but degrade as pitch and yaw angles grow, since the coupling terms become increasingly significant. This motivates the use of nonlinear controllers that account for the true coupled dynamics rather than a linearized approximation.

2.2 Geometric (SE(3)) Control

Geometric control formulates the attitude and position dynamics of a rigid body directly on the manifold of rigid-body motions, SE(3), rather than on a local Euclidean parameterization such as Euler angles. This avoids the singularities associated with Euler-angle representations and yields controllers with near-global stability properties, which is particularly attractive for large-angle manoeuvres of the type the TRMS is capable of. Geometric controllers of this type have been widely adopted in quadrotor and rotorcraft research as a robust alternative to linear PID control, and form the basis of the nonlinear control strategy targeted in this project.

2.3 Voice Interfaces and LLM-based Orchestration

Automatic speech recognition (ASR) systems such as OpenAI's Whisper and Google's Gemini API convert spoken audio into text with high accuracy across a range of acoustic conditions, making voice increasingly viable as a control-system input modality. Beyond transcription, large language model (LLM) based agent frameworks — such as LangGraph — allow a transcribed command to be parsed into a structured intent (for example, a target pitch and yaw trajectory) through multi-step reasoning, rather than through brittle keyword matching. Applying such agents to safety-critical hardware, however, requires an explicit human confirmation step, since LLM-based intent parsing is not guaranteed to be free of misinterpretation.

2.4 Human-in-the-Loop Safety

Human-in-the-Loop (HiL) design patterns insert a mandatory human confirmation step between an autonomous decision and its physical execution. In the context of this project, HiL is implemented as a simulation-backed approval gate: the AI agent's proposed trajectory is first rendered in a three-dimensional kinematic preview, and only after the operator explicitly authorizes the manoeuvre — by voice, through the mobile application — is the corresponding command transmitted to the physical actuators. This pattern is consistent with established safety practice for autonomous or semi-autonomous systems operating in proximity to people or valuable equipment.

CHAPTER 3: SYSTEM DESIGN AND METHODOLOGY

3.1 Design Philosophy

The system, named TwinTalk, follows a Human-in-the-Loop architecture in which the operator issues voice commands, the AI agent makes decisions, and the hardware executes only after explicit user authorization. This design philosophy was chosen deliberately: because the AI orchestrator's interpretation of a spoken command cannot be guaranteed to be error-free, and because the TRMS is a coupled nonlinear system in which an incorrect large-angle command could damage the hardware, no command is permitted to reach the physical actuators without a simulation-backed operator approval.

3.2 System Architecture

The end-to-end pipeline consists of six stages, summarised in Table 1 and illustrated conceptually in Figure 1.

Step	Stage	Description
1	Voice command to mobile app	Operator issues a voice command through the mobile application's microphone input.
2	Mobile app communicates with laptop	The command audio/transcript is sent to the laptop backend over Bluetooth or Wi-Fi.
3	AI agent decides if simulation is needed	The LangGraph AI agent analyses the parsed command and decides whether a simulation preview is required.
4	AI agent runs simulation and provides results	If required, the agent runs the MeshCat kinematic simulation and the results are presented to the operator.
5	User authorization via mobile app (voice)	The operator reviews the simulated preview and approves or rejects the manoeuvre by voice.
6	Laptop commands Raspberry Pi to execute	Upon approval, the laptop sends the validated command to the Raspberry Pi, which executes it on hardware.

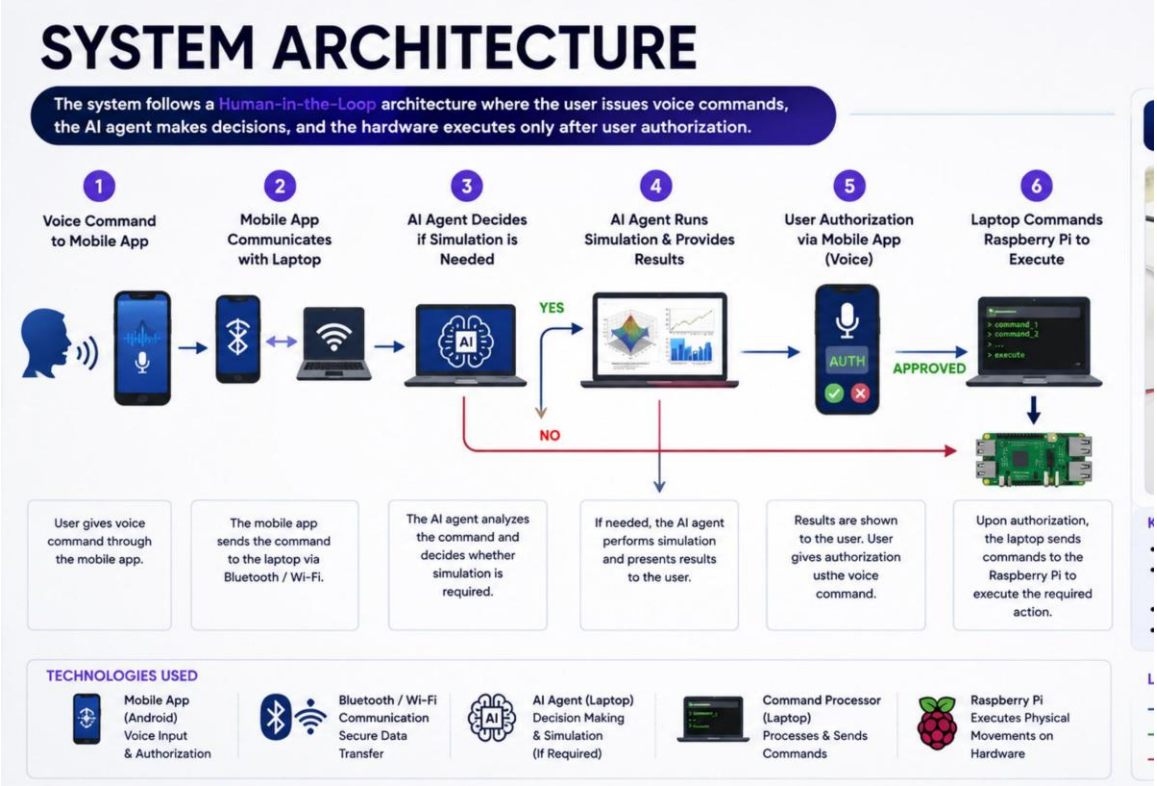


Figure 1: TwinTalk system architecture and Human-in-the-Loop pipeline

3.3 Core Deliverables

The project is organised around four core deliverables, listed in Table 2 below.

Deliverable	Description
Mobile Interface	A mobile application (Android / Flutter) providing voice input, connection status, trajectory approval and telemetry screens.
AI Orchestrator	A multi-agent LangGraph system that parses operator intent and generates specific trajectory commands.
3D Simulation / Safety Gate	A real-time MeshCat kinematic preview that requires explicit Human-in-the-Loop approval before any movement.
Hardware Control	Geometric (SE(3)) control algorithms and a Raspberry Pi interface that outputs precise PWM signals to the rotors.

Table 2: Core deliverables of the TwinTalk system

3.4 Technology Stack

Table 3 summarises the key software and technologies used across the mobile, AI, simulation and hardware-control layers of the system.

Component	Technology
Mobile Application	Android (Kotlin) initial prototype; Flutter used for the five-screen mid-term build
Speech Recognition (ASR)	OpenAI Whisper (target); Google Gemini API free tier (development)
AI Orchestration	Multi-agent LangGraph system (Python)
Simulation	MeshCat real-time kinematic preview
Numerical / Scientific Computing	Python — PyTorch, NumPy, SciPy
Control Design	MATLAB/Simulink for PID tuning and analysis
Communication	Bluetooth / Wi-Fi (app–laptop); WebSocket / HTTP (laptop backend)
Hardware Control	Raspberry Pi OS (Python) generating PWM signals to the rotors

Table 3: Key software and technology stack

3.5 Existing Hardware Setup

The project builds on an existing laboratory Twin Rotor platform (Figure 2), comprising a main rotor (R1), a tail rotor (R2) and a pivoted base, instrumented for pitch and yaw measurement and actuated through PWM-driven motor controllers. This hardware is treated as a fixed platform; the project's contribution is the voice-actuated control and orchestration layer built around it, together with the eventual replacement of the baseline PID controller with a nonlinear geometric controller.

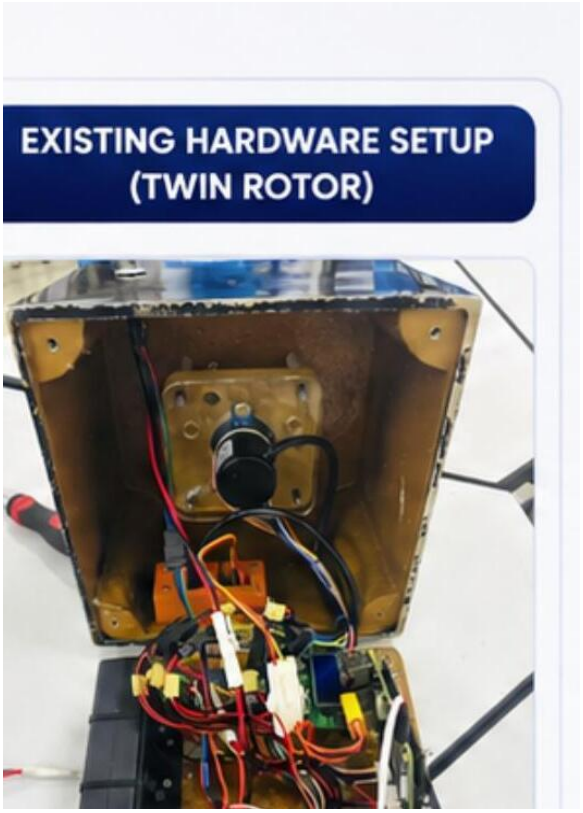


Figure 2: Existing Twin Rotor MIMO System (TRMS) hardware setup

Actual Twinrotor Hardware Setup

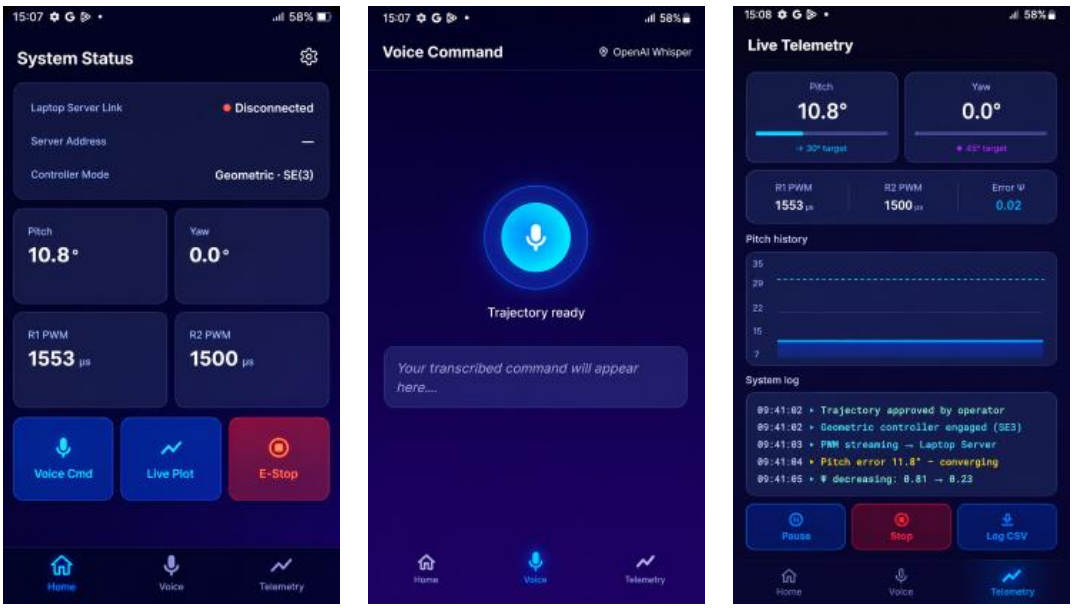
CHAPTER 4: IMPLEMENTATION

4.1 Mobile Application

A five-screen Flutter mobile application, TwinTalk, has been implemented and is fully connected to the laptop backend. The application captures the operator's voice command, displays connection and telemetry status, and hosts the Human-in-the-Loop approval step. The four functional screens, in addition to a launch screen, are summarised in Table 4.

Screen	Function
Connect / Status	Wi-Fi target selection and live pitch, yaw and PWM readout.
Voice Command	Gemini-based ASR transcription with parsed-intent display chips.
Trajectory Approval (HiL)	Two-dimensional preview gate that must be approved before any hardware write.
Telemetry	Live fl_chart plot of pitch versus target angle, together with a system log.

Table 4: Mobile application screens and functions



Home / System Status *Voice Command* *Live Telemetry*

Figure 5: Mobile application screens — launch, connection and status

4.2 Speech Recognition and Intent Parsing

Speech-to-text conversion was first implemented using the Gemini API free tier, in order to keep iteration cost-free while the rest of the pipeline was being built and debugged. The ASR layer was deliberately kept swappable as a design choice, and this proved necessary in practice: Gemini's transcription accuracy was found to be significantly lower than required, particularly on short imperative commands (e.g. "pitch to eighty degrees") and in the presence of background/rotor noise, frequently misparsing angle values and command keywords. The team therefore switched the ASR provider to OpenAI's Whisper model, which produced markedly more accurate and consistent transcriptions under the same conditions and is now used as the production ASR engine throughout the application (Figures 6 and 7).

Once transcribed, the operator's command is passed to a multi-agent orchestrator implemented using LangGraph. The orchestrator parses the operator's intent — for example, a target pitch or yaw angle, or a trajectory shape — and produces a structured command that can be either simulated for preview or, following approval, sent onward to the hardware layer.

4.3 Simulation and Human-in-the-Loop Safety Gate

A real-time three-dimensional kinematic preview of the twin-rotor body is rendered using MeshCat. When the AI agent determines that a simulation preview is required, it runs the kinematic simulation for the proposed trajectory and presents the result to the operator through the mobile application. The Human-in-the-Loop gate is enforced in software: the trajectory approval screen forms a mandatory checkpoint, and no command is transmitted to the Raspberry Pi unless the operator has explicitly approved it by voice. This mechanism has been implemented and validated as functioning correctly during the mid-term phase of the project.

4.3.1 Angle Safety Gate

In addition to the trajectory approval checkpoint, an angle safety gate was added directly to the Voice Command screen so that unsafe setpoints are caught before a trajectory preview is even generated. The requested pitch angle is checked against the physical range of the actuators as soon as the intent is parsed. If the requested angle falls between ±65° and ±85°, the actuators are approaching their mechanical limits, and

the app displays an amber "Actuators Saturated" warning indicating that the hardware implementation may not track the simulated preview accurately, while still allowing the operator to review and send the command (Figure 6). If the requested angle exceeds ±85°, it is outside the physically safe range altogether: the app blocks the command outright with a red "Angle exceeds safety threshold" error, disables the send action, and requires the operator to retry with a lower angle (Figure 7). This gate runs entirely on-device and does not depend on a response from the backend, so an unsafe angle is rejected immediately rather than after a round trip to the AI orchestrator.

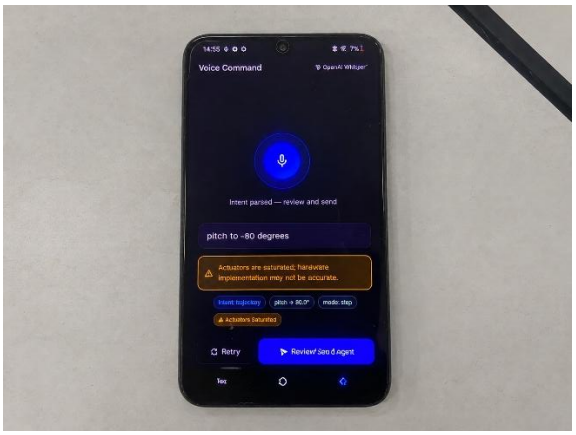


Figure 6: Voice Command screen showing the "Actuators Saturated" warning after a pitch command of -80° was parsed (within the ±65–±85° saturation band)

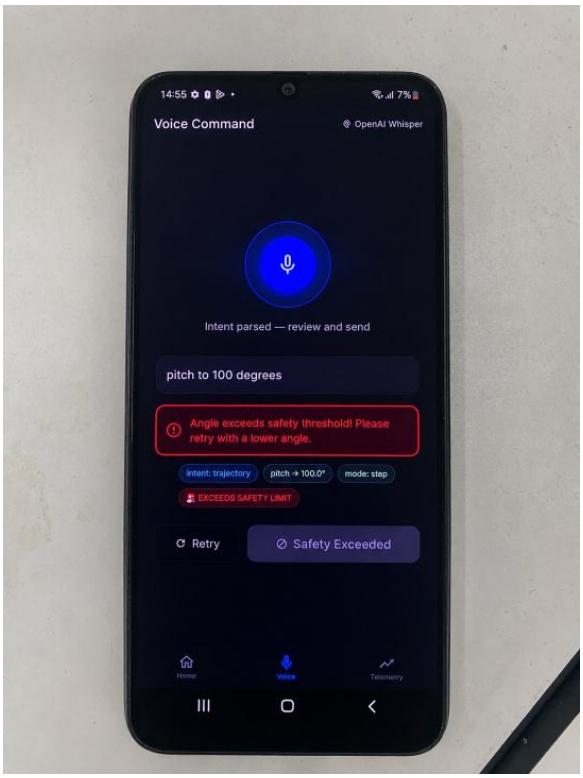


Figure 7: Voice Command screen showing the "Angle exceeds safety threshold" block for a pitch request of 100.0° (beyond ±85°)

4.4 Backend and Communication Layer

The mobile application communicates with a FastAPI backend running on a laptop over a WebSocket/HTTP connection. This connection was initially unreliable and required debugging of the connectivity handshake; it has since been resolved and tested, and voice commands issued through the mobile application now reliably reach the backend and are able to kick-start the MeshCat simulation. The backend, in turn, is responsible for forwarding approved commands to the Raspberry Pi, which will ultimately execute them on the physical rotors.

4.5 Hardware Control Layer

On the hardware side, a Raspberry Pi running Raspberry Pi OS is used to generate the pulse-width-modulated (PWM) signals that drive the main and tail rotor motor controllers. A linearized (Euler-angle) PID controller was designed first as the initial closed-loop baseline and characterised in MATLAB/Simulink ahead of being ported to Python on the Raspberry Pi. Simulation results showed that, while functional near hover, this linear baseline produced degraded tracking accuracy and increased overshoot as the commanded pitch angle approached the actuator's operating limits (the ±65°–±85° range flagged by the angle safety gate in Section 4.3.1), since the small-angle approximation underlying the linear error terms becomes progressively less valid at larger angles and is subject to gimbal-lock-like singularities near ±90°. To address this, the linear PID baseline was replaced with a nonlinear SO(3) geometric PID controller, in which the attitude error is computed directly on the rotation group rather than as a difference of Euler angles. In the MATLAB/Simulink comparison, this removed the singularity issue entirely, since the geometric error formulation has no gimbal-lock condition, and produced measurably more stable and accurate simulated tracking during large-angle manoeuvres close to the ±85° safety limit, where the linear controller had previously struggled most. The geometric controller also handles the coupling between pitch and yaw dynamics more faithfully than three independent linear loops, further improving tracking consistency across the operating envelope. This simulation-based comparison motivated the decision to carry the geometric controller forward as the baseline for physical hardware bring-up (Section 5.5); closed-loop performance data on the actual rig will be reported once that hardware integration is complete.

CHAPTER 5: EXPERIMENTS AND RESULTS

5.1 Mid-term Progress Summary

Table 5 summarises progress against the planned milestones at the mid-term stage of the fourteen-week project timeline shown in Figure 4 and Table 6.

Milestone	Due	Completion	Remarks
Literature review	Week 2	100%	TRMS control, geometric tracking, and Whisper/LLM orchestration surveyed.
System architecture	Week 3	100%	Human-in-the-Loop pipeline defined: App → AI Agent → Simulation → Raspberry Pi.
Mobile app (Flutter)	Week 5	100%	Five screens built: connect, status, voice, HiL approval, telemetry.
Basic 3D simulation	Week 5	85%	MeshCat kinematic preview running for pitch/yaw motion.
Voice → intent parsing	Week 5	85%	Using Gemini (free tier) in place of Whisper to control development cost.
App ↔ laptop backend link	Week 6	100%	WebSocket/HTTP connectivity issue identified and resolved.
Mobile app ↔ simulation pipeline	Week 6	100%	Voice command from the app successfully kick-starts the MeshCat simulation on the laptop.
Hardware integration (TRMS)	Week 9	10%	Connecting the laptop to the Raspberry Pi and verifying handshake protocols.

Table 5: Mid-term progress against planned milestones

5.2 Project Timeline

The project is organised into four phases across fourteen weeks, as shown in Table 6 and Figure 4.

Phase	Weeks	Tasks	Milestone
1. Project Kickoff	1–3	Literature review; GitHub repository setup; basic conceptualisation and interfaces.	Kickoff complete (end of Week 3)
2. Detailed Design	4–7	LangGraph agent routing; simulation environment; wireframing; mobile app skeleton.	Design complete (end of Week 7)
3. Initial Integration	8–9	Connect full end-to-end pipeline; test functionality; implement linearized PID controller.	Integration complete (end of Week 9)
4. Geometric Upgrade	10–14	Implement advanced geometric control algorithms; fine-tune physical robustness; finalize report.	Project complete (end of Week 14)

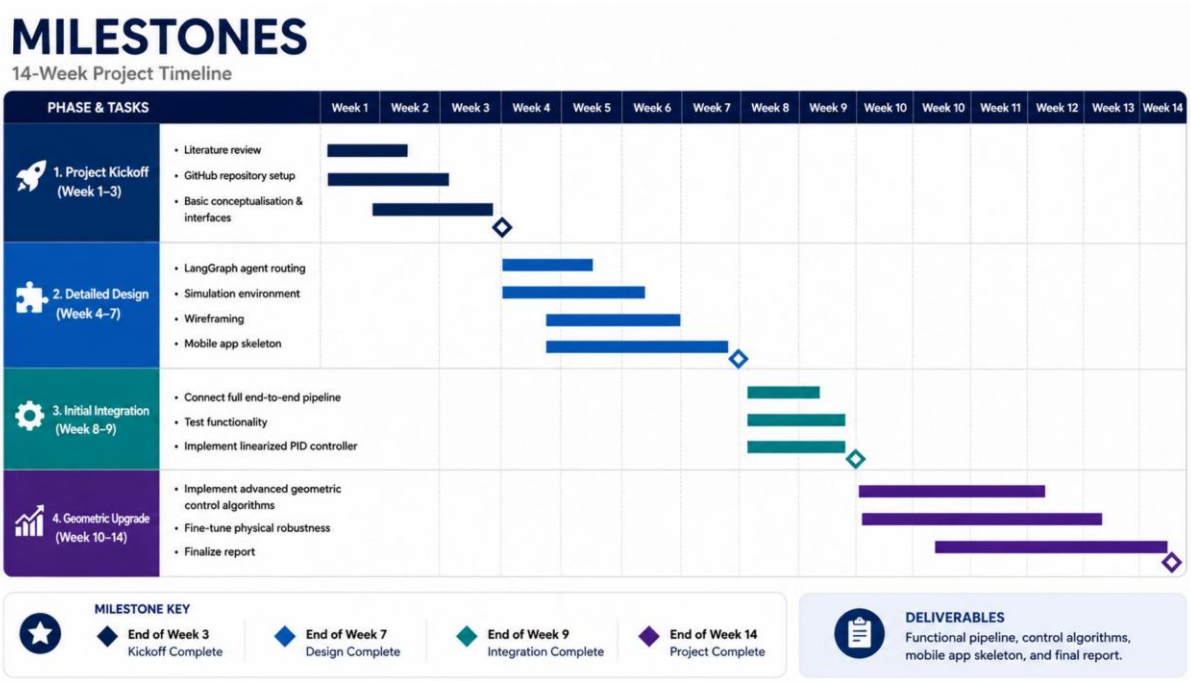


Figure 4: Fourteen-week project timeline (Gantt chart)

\

5.3 Justification of Achieved Results

The complete software pipeline is now operational end-to-end: a voice command issued through the mobile application is transcribed by the Gemini API, parsed by the AI agent into a structured trajectory intent, and automatically used to kick-start the MeshCat three-dimensional kinematic simulation on the laptop, with the Human-in-the-Loop approval gate enforced before any hardware command is issued (Figure 3).

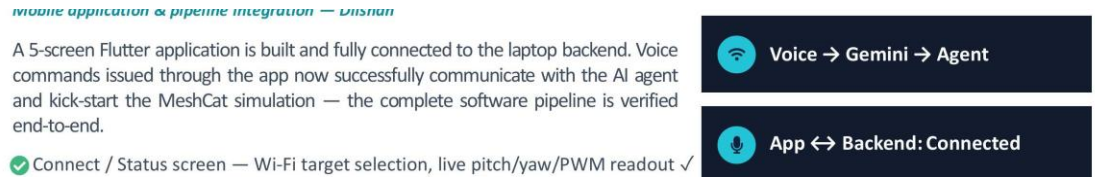


Figure 3: End-to-end voice-to-simulation software pipeline (Mobile App → Gemini API → AI Agent → MeshCat Simulation)

Specifically, the following results have been validated during this iteration of the project:

- The full voice-to-simulation pipeline has been verified end-to-end.
- The mobile application to laptop WebSocket/HTTP connection has been resolved and tested.
- MeshCat renders pitch and yaw motion live, and the Human-in-the-Loop approval gate is enforced before any simulated command could be forwarded to hardware.

The mobile application itself comprises five screens — connect/status, voice command, trajectory approval and telemetry, described in Section 4.1 — all of which have been implemented and tested against the laptop backend.

5.4 Engineering Trade-off: ASR Provider Selection

The Gemini API free tier was used first for speech-to-intent conversion, to keep the development iteration cost-free while the rest of the pipeline was being built. The ASR layer was deliberately kept swappable as a design choice, which allowed this decision to be revisited once real usage data was available: on laboratory testing, Gemini's transcription accuracy proved insufficient, with frequent misrecognition of numeric angle values and command keywords, especially with rotor/background noise present. The team switched to OpenAI's Whisper model, which gave a clear improvement in

transcription accuracy and reliability under the same test conditions, and Whisper has since been adopted as the final ASR provider for the system (Figures 6 and 7 show the "OpenAI Whisper" provider active on the Voice Command screen).

5.5 What Remains: Hardware Integration

With the software pipeline resolved, the remaining experimental work for the second half of the project centres on physical hardware integration, summarised in Table 7. As this work has not yet been carried out, no quantitative closed-loop performance data (e.g. settling time, overshoot, or tracking error for the PID or geometric controllers) is available at the time of writing; these results will be obtained and reported once hardware bring-up is complete.

Step	Stage	Description
1	Pi server	Deploy a WebSocket server on the Raspberry Pi; smoke-test PWM output with an oscilloscope.
2	Safety test	Confirm the HiL gate forces a simulation preview before any rotor command, and that the E-stop is enforced at the application level.
3	PID bring-up	Tune a linearized PID controller in MATLAB/Simulink; port it to Python on the Raspberry Pi and close the loop.
4	Geometric upgrade	Replace the PID controller with the SE(3) geometric controller for large-angle robustness; validate against simulation results.

Table 7: Hardware integration plan (Weeks 9–14)

In parallel with hardware bring-up, the remaining work includes: connecting to the physical TRMS by wiring the Raspberry Pi's PWM output to the main and tail rotors and confirming that the observed torque response matches the simulation; bringing up and confirming a linearized PID closed loop for pitch/yaw hold on the physical hardware as a sanity check, before deploying the SO(3) geometric controller (already selected as the target baseline following the MATLAB/Simulink comparison in Section 4.5) on the physical rig; and, finally, documenting all results and recording an end-to-end voice-to-rotor demonstration for the final report. The ASR provider trade-off (Section 5.4) has already been resolved in favour of Whisper and does not require further benchmarking.

5.6 Yaw-Axis Hardware Limitation and Justification for Pitch-Only Control

During bring-up and testing of the physical Twin Rotor rig, it was observed that the yaw axis exhibited random, uncommanded rotation even when the twin-rotor body was otherwise mechanically at rest and no yaw command had been issued. Figure 8 shows a representative terminal log captured from the Raspberry Pi during this test: Encoder2, which measures the pitch axis, reports a constant reading of -2397 ticks throughout the log, confirming that the pitch axis was correctly holding a stable position. Encoder1, which measures the yaw axis, is seen counting continuously and unpredictably over the same interval (from approximately -7669 to -8599 ticks) despite the rig being physically stationary in yaw.

This behaviour was traced to a hardware issue on the yaw axis of the Twin Rotor platform itself, rather than to a fault in the control software or the encoder-reading logic, since the drift was present consistently and independent of any commanded PWM signal on that axis. Because the yaw-axis readings could not be trusted to represent the true physical orientation of the rig, closed-loop control based on this feedback would have been unreliable and potentially unsafe. For this reason, the yaw axis was excluded from closed-loop testing and validation for this iteration of the project, and all controller tuning, comparison and validation work — including the geometric SO(3) PID controller described in Section 4.5 — was carried out and reported using the pitch axis only, for which the encoder feedback remained stable and trustworthy.

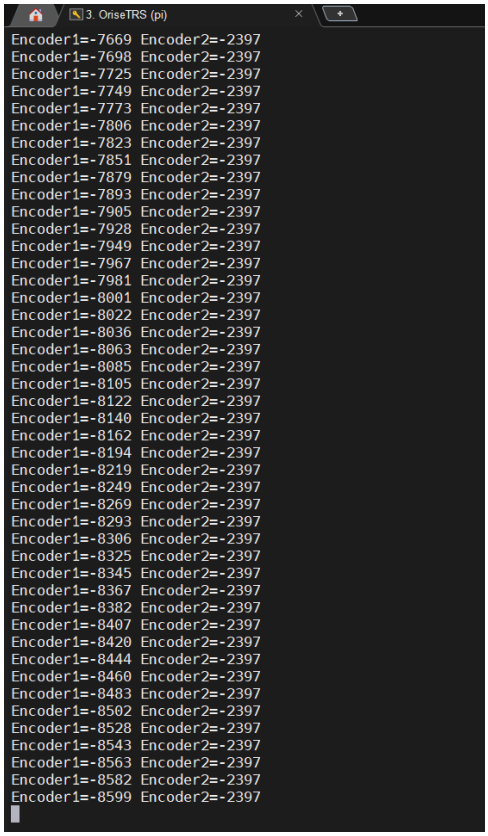


Figure 8: Raspberry Pi terminal log showing the yaw axis (Encoder1) drifting randomly while the pitch axis (Encoder2) remains constant at -2397 ticks, confirming a yaw-axis hardware fault

CHAPTER 6: CONCLUSION AND RECOMMENDATIONS

6.1 Conclusion

At the mid-term stage of this project, the team has successfully designed and implemented a complete, end-to-end software pipeline for voice-actuated control of a Twin Rotor platform, comprising a five-screen mobile application, a speech-to-intent module, a multi-agent AI orchestrator, and a three-dimensional simulation-backed Human-in-the-Loop safety gate. All planned software milestones up to Week 6 have been completed, and the mobile-to-laptop communication layer, previously a source of connectivity issues, has been resolved and tested. Physical hardware integration remains the primary outstanding work, currently at an early stage of connecting the Raspberry Pi to the twin-rotor actuators.

6.2 Impact Analysis

Table 8 summarises the anticipated economic, social and environmental impact of the project.

Dimension	Analysis
Economic	Built entirely on an open-source Python stack, eliminating licensing fees. The project demonstrates a clear return on investment for AI-augmented infrastructure and provides skills that are directly transferable to the commercial UAV industry.
Social	Hands-free operation significantly reduces operator cognitive load in complex scenarios. The intuitive mobile interface and voice-command capability drastically lower the barrier to entry for operating aerospace-representative hardware.
Environmental	Efficient, well-optimised code reduces power consumption, extending battery life. By testing extensively in a digital simulator before committing to physical actuation, the system avoids many real-world test cycles, reducing wear on equipment and lowering the total energy used over the course of the project.

Table 8: Impact analysis — economic, social and environmental

6.3 Recommendations

Based on progress to date, the team recommends prioritising the following actions in the remaining phase of the project:

- Complete the Raspberry Pi PWM smoke-test and handshake verification before attempting closed-loop control, to isolate hardware faults from control-design faults.
- Bring up and validate the linearized PID controller on physical hardware before introducing the geometric controller, so that the nonlinear upgrade can be benchmarked against a known working baseline.
- Validate the chosen Whisper ASR provider under realistic rotor-noise conditions on the physical rig early in the remaining timeline, to confirm the accuracy improvement observed during development carries over to the lab environment.
- Reserve adequate time in the final two weeks for full-pipeline demonstration and report-writing, rather than treating these as an afterthought to the control-algorithm work.

CHAPTER 7: FUTURE SCOPE

Beyond the immediate remaining work described in Chapter 5, several extensions could build on the TwinTalk platform once the core voice-to-rotor pipeline is fully validated on hardware. The nonlinear SE(3) geometric controller, once implemented and tuned, could be extended to track more complex time-varying trajectories rather than static setpoints, allowing the operator to issue commands such as continuous sweeping or figure-eight patterns by voice. The AI orchestrator could also be extended to support multi-turn conversational correction, allowing the operator to refine a proposed trajectory verbally (for example, “a bit less pitch”) rather than restating the full command.

On the safety side, the current Human-in-the-Loop gate could be augmented with automated anomaly detection, flagging commands that fall outside historically safe operating envelopes for additional scrutiny even after operator approval. Finally, once the ASR benchmarking is complete and a final provider is selected, the system could be evaluated for robustness to a wider range of accents, background noise conditions and network latency, extending its applicability beyond the controlled laboratory environment in which it has been developed and tested.

REFERENCES

1. Quanser Inc., “Twin Rotor MIMO System — User Manual,” Quanser Consulting Inc., Markham, Ontario, Canada.
2. LangChain AI, “LangGraph Documentation,” [Online]. Available: https://langchain-ai.github.io/langgraph/
3. MeshCat contributors, “MeshCat: a WebGL-based 3D visualizer for Python and Julia,” [Online]. Available: https://github.com/meshcat-dev/meshcat-python
4. Google, “Gemini API Documentation,” [Online]. Available: https://ai.google.dev/gemini-api/docs
5. OpenAI, “Whisper: Robust Speech Recognition via Large-Scale Weak Supervision,” [Online]. Available: https://github.com/openai/whisper
6. Raspberry Pi Foundation, “Raspberry Pi OS Documentation,” [Online]. Available: https://www.raspberrypi.com/documentation/
7. Twin rotor user manual

APPENDICES

Appendix A: Project Repository

The complete source code for the mobile application, AI orchestrator, simulation environment and hardware control scripts are version-controlled and publicly accessible at: https://github.com/Dilshan-Prince/ME325-Voice-Command-Twin-rotor-

Appendix B: Team Roles

- E/21/100 Dilshan — Mobile application development and app–backend pipeline integration.
- E/21/205 Shathurshiga — AI orchestration and full pipeline (simulation-to-app) integration.
- E/21/338 Riswan — AI orchestration and full pipeline (simulation-to-app) integration.

Appendix C: Supervisor

Dr. D. H. S. Maithripala, Department of Mechanical Engineering, Faculty of Engineering, University of Peradeniya.